

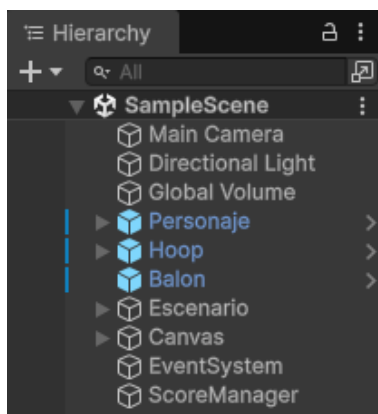
	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRÁFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>Motores de Física, Colisiones</i>				
NÚMERO DE PRÁCTICA:	01	AÑO LECTIVO:	2026A	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	22/04/26	HORA DE PRESENTACIÓN	12:20:00		
INTEGRANTE (s): Venero Guevara Christian Henry				NOTA:	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Repositorio: https://github.com/Cvenero/Laboratorios_Computacion-Grafica/tree/main/Laboratorio_01 El presente informe muestra el proceso de desarrollo de un videojuego de basketball en 3D utilizando el motor de juego Unity. Este trabajo muestra temas de desarrollo incluyendo física de objetos, detección de colisiones y sistema de puntuación. El juego permite controlar un personaje que “driblea”, apunta y lanza un balón hacia un aro cuadrado. El sistema detecta automáticamente cuando el balón pasa por el aro y suma puntos al marcador visible en pantalla. 1. Objetivos Específicos • Implementar el movimiento del personaje con el sistema New Input System de Unity. • Programar la mecánica de dribleo, apuntado y lanzamiento del balón. • Configurar la física del balón con Rigidbody y PhysicMaterial para rebotes. • Detectar canastas mediante Trigger Colliders y el método OnTriggerEnter. • Crear una interfaz con TextMeshPro para mostrar la puntuación. 2. Estructura de la Escena La escena del proyecto se organizó con los siguientes GameObjects en la Hierarchy de Unity: • Main Camera: cámara principal con vista top-down.

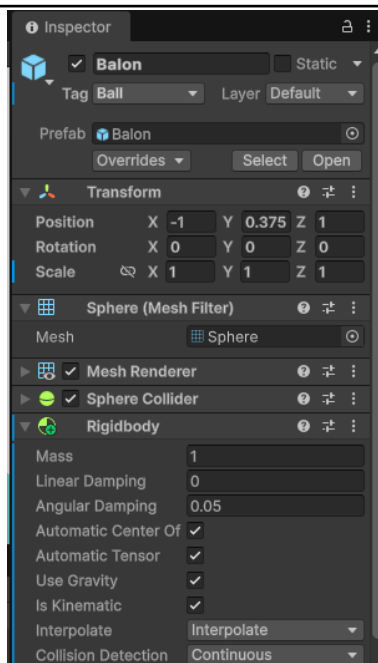
- Personaje: objeto del jugador con scripts de control.
- Hoop: grupo que contiene los 4 cubos del aro, soporte, tablero y marcador (trigger).
- Balon: esfera con Rigidbody y tag 'Ball'.
- Escenario: plano del suelo con Mesh Collider.
- Canvas: interfaz de usuario con ScoreText (TextMeshPro).
- ScoreManager: objeto vacío con el script de puntuación.



3. Configuración del Balón

El balón fue configurado como una Sphere con los siguientes componentes y ajustes:

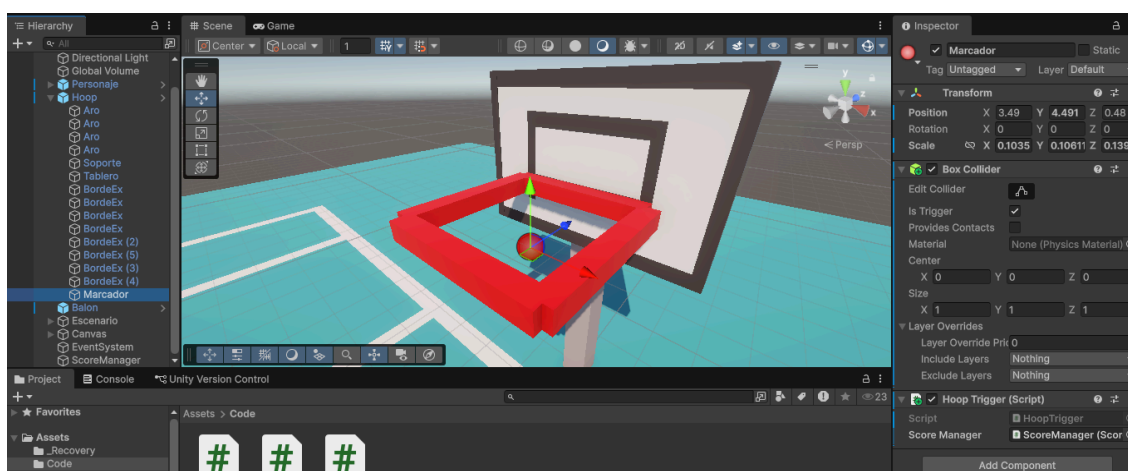
- Rigidbody: Mass=1, Collision Detection=Continuous, Interpolate=Interpolate.
- PhysicMaterial: Bounciness=0.5 para rebotes.
- Tag: 'Ball' para ser detectado por el trigger del aro.



4. Configuración del Aro

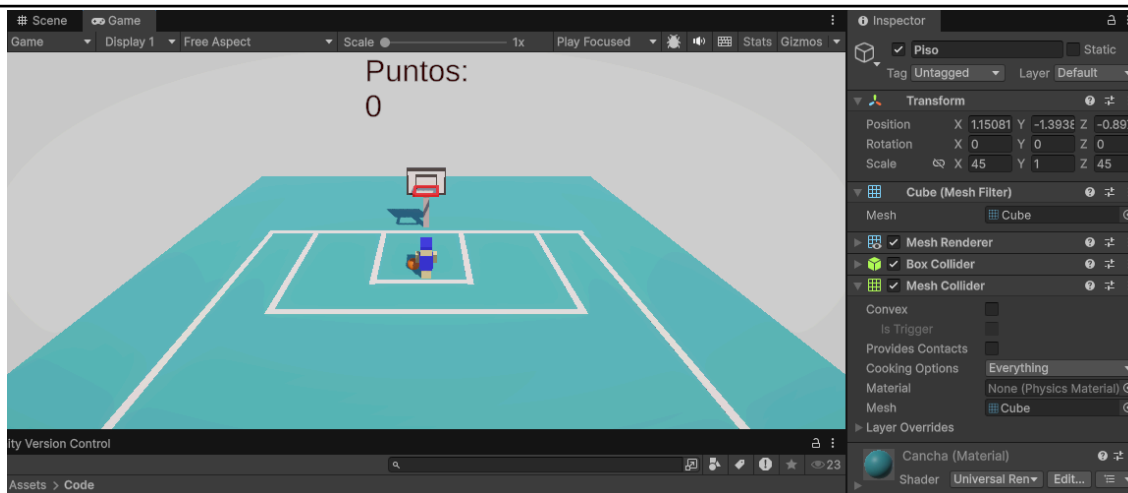
El aro fue construido con 4 Cubos alargados formando un cuadrado. Dentro del objeto padre Hoop se creó un GameObject vacío llamado Marcador que cumple dos funciones: ser el punto de apuntado del personaje al lanzar, y ser el trigger detector de canastas.

- Box Collider con Is Trigger activado: Size X=1, Y=1, Z=1
- Script HoopTrigger.cs adjunto al Marcador.



5. Escenario

El suelo del escenario utiliza un Cube con Mesh Collider para que el balón rebote correctamente.



6. Código Fuente

Cada script se presenta dividido en fragmentos con una explicación detallada de que hace cada parte del código.

6.1. BasketController.cs - Control del Personaje

Script principal del juego. Controla el movimiento, el dribbleo, el apuntado, el lanzamiento y la recuperación automática del balón.

Variables públicas y privadas

Se declaran todas las variables del script. Las públicas (public) aparecen en el Inspector de Unity y permiten conectar objetos de la escena arrastrándolos. Las privadas (private) solo existen internamente en el script.

CSharp

```
public float MoveSpeed = 10;           // Velocidad de desplazamiento
public Transform ball;                 // Referencia al balón en la escena
public Transform arms;                 // Referencia a los brazos del personaje
public Transform posOverHead;          // Posición del balón al apuntar (sobre la
cabeza)
public Transform posDribble;            // Posición del balón al driblear (frente al
personaje)
public Transform target;               // El Marcador del aro (punto de apuntado)
public float shootForce = 10f;         // Fuerza con la que se lanza el balón

private Rigidbody rb;                 // Componente de física del balón
private bool inHands = true;           // true = balón en manos, false = balón volando
private float timeSinceShot = 0f;      // Tiempo transcurrido desde el lanzamiento
private float pickupDelay = 0.8f;      // Segundos de espera antes de intentar recoger
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

Método Start()

Start() se ejecuta una sola vez al iniciar el juego. Se usa para obtener el componente Rigidbody del balón y dejarlo en modo cinemático (sin física activa) porque al inicio el balón está en manos del personaje.

CSharp

```
void Start()
{
    rb = ball.GetComponent<Rigidbody>(); // Obtiene el Rigidbody del balón
    rb.isKinematic = true;              // Desactiva la física al inicio
}
```

GetComponent<Rigidbody>() busca el componente Rigidbody en el objeto balón. isKinematic = true significa que Unity no aplica gravedad ni fuerzas al balón, permitiendo que el script lo mueva manualmente.

Movimiento del personaje (WASD)

Dentro de Update() (que se ejecuta cada frame) se detectan las teclas presionadas y se calcula la dirección de movimiento. El personaje se mueve y rota hacia esa dirección.

CSharp

```
Vector3 direction = Vector3.zero; // Sin movimiento por defecto
if (Keyboard.current.wKey.isPressed) direction.z += 1; // Adelante
if (Keyboard.current.sKey.isPressed) direction.z -= 1; // Atras
if (Keyboard.current.aKey.isPressed) direction.x -= 1; // Izquierda
if (Keyboard.current.dKey.isPressed) direction.x += 1; // Derecha

if (direction != Vector3.zero)
{
    transform.position += direction * MoveSpeed * Time.deltaTime;
    transform.LookAt(transform.position + direction); // Girar hacia donde se
    mueve
}
```

Time.deltaTime hace que el movimiento sea igual de rápido sin importar los FPS del juego. Sin el personaje se movería más rápido en computadoras potentes. LookAt() rota el personaje para que mire hacia la dirección en que se mueve.

Dribleo y apuntado (Espacio)

Cuando el balón está en manos (inHands = true), se anima el dribleo usando una función matemática. Al mantener presionado la tecla "Espacio", el personaje levanta el balón sobre la cabeza y mira hacia el aro.

CSharp

```
if (inHands)
{
    rb.isKinematic = true; // Mientras esta en manos, sin física
}
```

```
if (Keyboard.current.spaceKey.isPressed)
{
    // Apuntando: balon sobre la cabeza, personaje mira al aro
    ball.position = posOverHead.position;
    arms.localEulerAngles = Vector3.right * 180;
    transform.LookAt(target.position);
}
else
{
    // Dribleo: el balon sube y baja con Mathf.Sin
    ball.position = posDribble.position +
        Vector3.up * Mathf.Abs(Mathf.Sin(Time.time * 5));
    arms.localEulerAngles = Vector3.zero;
}
if (Keyboard.current.fKey.wasPressedThisFrame)
    Shoot(); // Lanzar al presionar F
}
```

Mathf.Sin genera un valor que oscila entre -1 y +1 continuamente. Mathf.Abs lo convierte a positivo (0 a 1), creando el efecto de rebote del dribleo. Time.time * 5 controla la velocidad de la animación.

Recuperación automática del balón

Después de lanzar el balón, el personaje puede recuperarlo acercándose a él. El pickupDelay evita que lo recoja inmediatamente en el aire justo después de lanzarlo.

CSharp

```
if (!inHands) // Solo cuando el balon fue lanzado
{
    timeSinceShot += Time.deltaTime; // Cuenta el tiempo transcurrido
    if (timeSinceShot > pickupDelay) // Espera 0.8 segundos primero
    {
        float distance = Vector3.Distance(transform.position, ball.position);
        bool ballSlowed = rb.linearVelocity.magnitude < 3f; // Balon casi quieto

        if (distance < 2.5f && ballSlowed) // Cerca y lento = recoger
            PickBall();
    }
}
```

Vector3.Distance mide la distancia entre el personaje y el balón. rb.linearVelocity.magnitude mide la velocidad total del balón. Si ambas condiciones se cumplen (está cerca y casi no se mueve), el personaje lo recoge automáticamente.

Métodos PickBall() y Shoot()

PickBall() resetea el balón a las manos del personaje. Shoot() activa la física y lanza el balón hacia adelante y arriba.

CSharp

```
void PickBall()
{
    inHands = true;
    rb.linearVelocity = Vector3.zero;    // Para el movimiento del balon
    rb.angularVelocity = Vector3.zero;   // Para la rotacion del balon
    rb.isKinematic = true;              // Desactiva la fisica
    ball.position = posDribble.position; // Regresa el balon a la mano
}
void Shoot()
{
    inHands = false;
    timeSinceShot = 0f;                // Reinicia el temporizador
    rb.isKinematic = false;            // Activa la fisica
    Vector3 shootDirection = transform.forward + Vector3.up * 1f; //Parabola
    rb.linearVelocity = shootDirection * shootForce; // Aplica velocidad
}
```

transform.forward es el vector que apunta hacia donde mira el personaje. Al sumarle Vector3.up * 1f se agrega un componente hacia arriba, creando la trayectoria curva del lanzamiento. Multiplicar por shootForce controla qué tan fuerte se lanza.

6.2. HoopTrigger.cs - Deteccion de Canastas

Script adjunto al objeto Marcador (trigger invisible dentro del aro). Detecta cuando el balón pasa por el aro y suma puntos.

Variable de referencia

Solo necesita una referencia al ScoreManager para poder llamar al método AddPoints cuando se detecte una canasta.

CSharp

```
public ScoreManager scoreManager; // Referencia al gestor de puntuacion
```

Esta variable es pública para poder arrastrar el objeto ScoreManager desde la Hierarchy de Unity al campo en el Inspector.

Deteccion del trigger

Unity llama automáticamente a OnTriggerEnter() cada vez que un objeto con Rigidbody entra en la zona del trigger. Se verifica que sea el balón y que venga de arriba hacia abajo.

CSharp

```
void OnTriggerEnter(Collider other)
{
    // Verifica que el objeto que entro tiene el tag 'Ball'
```

```
if (other.CompareTag("Ball"))
{
    Rigidbody rb = other.GetComponent<Rigidbody>();

    // rb.linearVelocity.y < 0 significa que el balón va hacia abajo
    // Esto evita contar rebotes que suban por el aro accidentalmente
    if (rb != null && rb.linearVelocity.y < 0)
    {
        scoreManager.AddPoints(1); // Suma 1 punto
    }
}
```

CompareTag() es más eficiente que comparar strings directamente. Verificar que la velocidad en Y sea negativa ($rb.linearVelocity.y < 0$) garantiza que solo se cuente cuando el balón entra por arriba del aro, no si rebota desde abajo.

6.3. ScoreManager.cs - Gestión de Puntuación

Script que lleva la cuenta de los puntos y actualiza el texto en pantalla. Se adjunta a un GameObject vacío en la escena.

Variable del texto y puntuación

Declara la referencia al texto de pantalla y la variable que guarda los puntos acumulados.

CSharp

```
public TextMeshProUGUI scoreText; // El texto de la pantalla (TextMeshPro)
private int score = 0; // Puntos acumulados, inicia en 0
```

TextMeshProUGUI es el tipo de componente de texto que usa TextMeshPro. La variable score es privada porque sólo este script debe modificarla. Otros scripts la modifican indirectamente llamando al método AddPoints().

Método AddPoints()

Método público que otros scripts llaman para sumar puntos. Actualiza la variable score y el texto visible en pantalla.

CSharp

```
public void AddPoints(int points)
{
    score += points; // Suma los puntos
    scoreText.text = "Puntos: " + score; // Actualiza el texto en pantalla
}
```

Este método es público para que HoopTrigger.cs pueda llamarlo desde otro script. La concatenación "Puntos: " + score convierte el número entero a texto automáticamente.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

7. Resultados

Al ejecutar el proyecto se obtuvieron los siguientes resultados funcionales:

- El personaje se mueve correctamente en las 4 direcciones con WASD y gira hacia la dirección de movimiento.
- El balón realiza un movimiento de dribleo animado usando Mathf.Sin mientras está en manos del personaje.
- Al mantener presionado la tecla “Espacio”, el personaje levanta el balón y apunta automáticamente hacia el aro.
- Al presionar F se lanza el balón con física: velocidad, gravedad y rebotes en el suelo.
- El balón rebota correctamente gracias al Mesh Collider del suelo y el Physic Material.
- Cuando el balón pasa por el trigger en dirección descendente, el marcador suma 1 punto.
- El personaje recoge el balón automáticamente al acercarse una vez que su velocidad baja de 3 unidades/segundo.

II. SOLUCIÓN DEL CUESTIONARIO

III. CONCLUSIONES

El desarrollo de este laboratorio permitió comprender y aplicar los conceptos fundamentales del desarrollo de videojuegos en Unity. Se logró implementar un juego funcional de basketball 3D que integra varios sistemas trabajando en conjunto.

- Se comprendió el uso del componente Rigidbody.
- Se aprendió la diferencia entre un Collider normal (barrera física) y un Trigger (zona de detección), aplicándolo para detectar canastas.
- El New Input System de Unity ofrece mayor control y claridad en la detección de entrada del teclado respecto al sistema clásico.
- El uso de TextMeshPro permite crear interfaces de usuario.
- La depuración de errores como el balón hundiéndose en el suelo o no recuperarse automáticamente fortaleció la comprensión del motor físico de Unity.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Unity Technologies, “Unity 6.4 user manual”, Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com>. [Consultado: 22-abr-2026].

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

- [2] Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/Packages/com.unity.inputsystem>. [Consultado: 22-abr-2026].
- [3] "TextMesh Pro Documentation", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/Manual/com.unity.textmeshpro.html>. [Consultado: 22-abr-2026].
- [4] Unity Technologies, "Rigidbody component reference", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/Manual/class-Rigidbody.html>. [Consultado: 22-abr-2026].
- [5] Unity Technologies, "Colliders - Unity Manual", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/es/2018.4/Manual/CollidersOverview.html>. [Consultado: 22-abr-2026].
- [6] Unity Technologies, "Módulo Triggers", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/es/2021.1/Manual/PartSysTriggersModule.html>. [Consultado: 22-abr-2026].